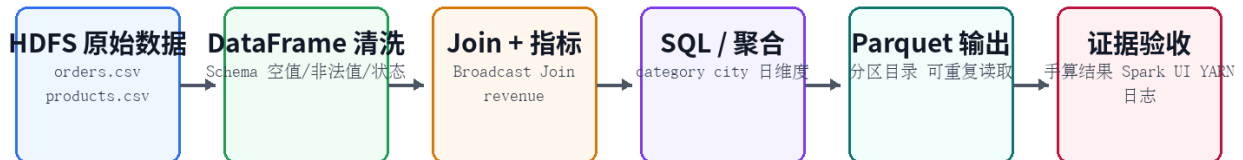


完整 PySpark 项目实战：从 HDFS 原始数据到可验证结果

目标：把前 18 讲的 HDFS、YARN、DataFrame、SQL、Join、Shuffle、cache、Parquet、Spark UI 与故障排查真正串成一次可运行闭环。

第19讲项目闭环：从 HDFS 原始数据到可验证分析结果



核心判断：Job 显示 SUCCESS 只证明“跑完了”，不证明“算对了”。

本讲使用三类证据：业务手算答案 + 数据质量计数 + Spark/YARN 执行证据。

本讲不是“照着敲代码”。你需要完成四个闭环：业务结果闭环、数据质量闭环、分布式执行闭环、故障排查闭环。

1. 项目场景：小型电商销售分析

假设 HDFS 中每天进入两份数据：订单事实数据 orders.csv，以及商品维表 products.csv。现在需要构建一个离线分析 Job：只统计已经支付、字段合法、商品维度可匹配的订单，并输出类别销售额、城市销售额与清洗后的明细。

1.1 业务问题

- 问题 A：各商品类别的销售额和有效订单数是多少？
- 问题 B：各城市的销售额是多少？
- 问题 C：原始 13 条订单中，有多少条因为状态、字段非法或维表无法匹配而被排除？
- 问题 D：最终数据以 Parquet 写回 HDFS 后，能否再次被 Spark 正确读取？

科研/工程思维：先写清楚“正确答案应该是什么”，再写程序。否则程序能跑通，你也不知道是否算对。

1.2 两张输入表

order_id	user_id	product_id	city	order_date	quantity	unit_price	status
O001	U001	P001	Beijing	2026-09-01	1	2999	PAID
O002	U002	P003	Shanghai	2026-09-01	2	99	PAID
O003	U003	P005	Beijing	2026-09-01	3	59	CANCELLED
O004	U001	P004	Beijing	2026-09-02	1	199	PAID
O005	U004	P002	Guangzhou	2026-09-02	1	5999	PAID
O006	U005	P003	Shanghai	2026-09-02	1	99	REFUND
O007	U006	P005	Beijing	2026-09-02	2	59	PAID
O008	U007	P001	Shenzhen	2026-09-03	1	2999	PAID
O009	U008	P003	Shanghai	2026-09-03	5	99	PAID
O010	U009	P004	Guangzhou	2026-09-03	2	199	PAID
O011	U010	P999	Beijing	2026-09-03	1	100	PAID
O012	U011	P002	Shanghai	2026-09-03	0	5999	PAID
O013	U012	P003	Beijing	2026-09-04	2	abc	PAID

product_id	product_name	category
P001	Phone	Digital
P002	Laptop	Digital
P003	Mouse	Accessory
P004	Keyboard	Accessory
P005	Book	Education

2. 先手算：最终正确结果应该是什么？

有效规则：status=PAID；quantity>0；unit_price 可解析且>0；product_id 必须能在 products 表中匹配。revenue = quantity × unit_price。

数据质量漏斗：每一步都要能解释“为什么少了这些记录”



阶段	剩余条数	被排除的记录
原始 orders	13	-
只保留 PAID	11	O003(CANCELLED)、O006(REFUND)
字段合法	9	O012(quantity=0)、O013(price=abc)
商品维表可匹配	8	O011(product_id=P999)
最终分析集	8	进入聚合与输出

手算后的两个核心结果：

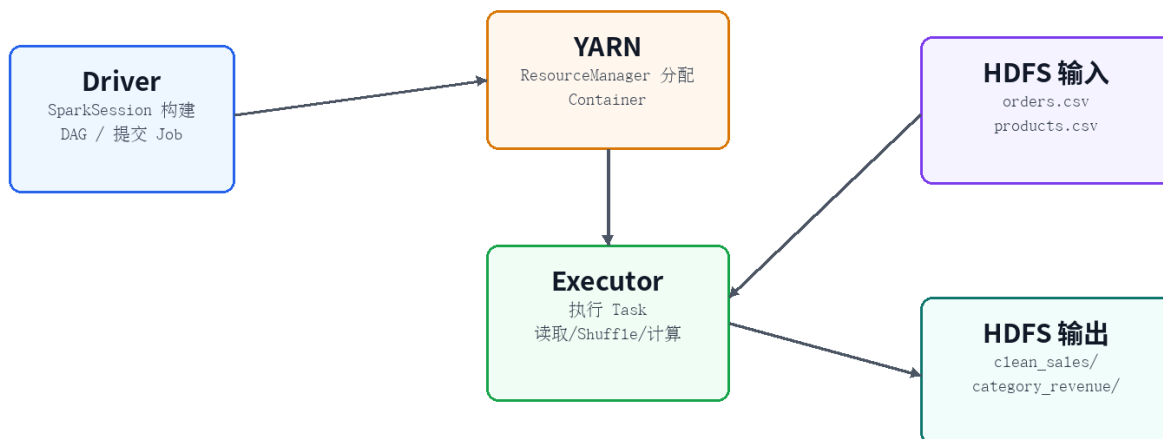
category	revenue	有效订单数
Digital	11997	3
Accessory	1290	4
Education	118	1

city	revenue
Guangzhou	6397
Beijing	3316
Shenzhen	2999
Shanghai	693

总销售额应为 13405。如果 Spark 输出不是这些数，不要先怀疑 Spark；先检查清洗、Join 和 revenue 计算。

3. 运行架构：这次到底有哪些系统在协作？

运行架构：PySpark 程序、YARN 与 HDFS 如何协作



client 模式：Driver 在 spark-submit 所在进程；Executor 仍由 YARN Container 承载。

你写的是 PySpark 程序，但真正参与执行的至少有四层：Driver 负责构建和调度；YARN 分配 Container；Executor 执行 Task；HDFS 提供输入和保存输出。

3.1 每次上机先做环境检查

在 VS Code 远程终端执行：

```
# 1) 检查 Java / Hadoop / Spark
java -version
hadoop version
spark-submit --version

# 2) 检查 HDFS 与 YARN 是否正常运行
start-dfs.sh
start-yarn.sh

# 3) 检查进程
jps
hdfs dfsadmin -report
yarn node -list

# 4) 检查 Spark 是否依赖 Hadoop/YARN 配置
echo $HADOOP_CONF_DIR
echo $YARN_CONF_DIR
```

如果 'yarn node -list' 看不到 RUNNING 节点，先不要运行 Spark。资源管理层没有就绪，后面出现的错误通常只是连锁反应。

4. 准备实验数据并上传 HDFS

为了让你能直接复现，本讲用 Shell here-document 一次性创建两个 CSV 文件。

创建本地 CSV：

```
mkdir -p ~/spark19
cd ~/spark19

cat > orders.csv <<'EOF'
order_id,user_id,product_id,city,order_date,quantity,unit_price,status
0001,U001,P001,Beijing,2026-09-01,1,2999,PAID
```

```
0002,U002,P003,Shanghai,2026-09-01,2,99,PAID
0003,U003,P005,Beijing,2026-09-01,3,59,CANCELLED
0004,U001,P004,Beijing,2026-09-02,1,199,PAID
0005,U004,P002,Guangzhou,2026-09-02,1,5999,PAID
0006,U005,P003,Shanghai,2026-09-02,1,99,REFUND
0007,U006,P005,Beijing,2026-09-02,2,59,PAID
0008,U007,P001,Shenzhen,2026-09-03,1,2999,PAID
0009,U008,P003,Shanghai,2026-09-03,5,99,PAID
0010,U009,P004,Guangzhou,2026-09-03,2,199,PAID
0011,U010,P999,Beijing,2026-09-03,1,100,PAID
0012,U011,P002,Shanghai,2026-09-03,0,5999,PAID
0013,U012,P003,Beijing,2026-09-04,2,abc,PAID
EOF
```

```
cat > products.csv <<'EOF'
product_id,product_name,category
P001,Phone,Digital
P002,Laptop,Digital
P003,Mouse,Accessory
P004,Keyboard,Accessory
P005,Book,Education
EOF
```

```
# 验证数据
wc -l orders.csv products.csv
head orders.csv
cat products.csv
```

上传并验证 HDFS：

```
hdfs dfs -mkdir -p /user/hadoop/spark19/input
hdfs dfs -put -f orders.csv products.csv /user/hadoop/spark19/input/
```

```
hdfs dfs -ls /user/hadoop/spark19/input
hdfs dfs -cat /user/hadoop/spark19/input/orders.csv | head
```

5. 第一步代码：SparkSession、Schema 与读取 HDFS

这一阶段对应第15讲的 DataFrame 与 Schema。我们不使用 inferSchema，而是显式声明类型，使 “ quantity 应该是整数、unit_price 应该是数值 ” 成为程序契约。

```
from pyspark.sql import SparkSession
from pyspark.sql import functions as F
from pyspark.sql.functions import col, broadcast
from pyspark.sql.types import (
    StructType, StructField, StringType, IntegerType, DoubleType
)

spark = (SparkSession.builder
        .appName("Lecture19SalesETL")
        .getOrCreate())

orders_schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("user_id", StringType(), True),
    StructField("product_id", StringType(), True),
    StructField("city", StringType(), True),
    StructField("order_date", StringType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("unit_price", DoubleType(), True),
    StructField("status", StringType(), True),
])

products_schema = StructType([
    StructField("product_id", StringType(), True),
    StructField("product_name", StringType(), True),
    StructField("category", StringType(), True),
])

orders = (spark.read.option("header", True)
        .schema(orders_schema)
        .csv("hdfs:///user/hadoop/spark19/input/orders.csv"))

products = (spark.read.option("header", True)
        .schema(products_schema)
        .csv("hdfs:///user/hadoop/spark19/input/products.csv")
        .dropDuplicates(["product_id"]))

orders.printSchema()
orders.show(truncate=False)
```

注意 O013 的 `unit\_price=abc`。在显式 DoubleType 下，这个字段不能被解析成合法数值，后续会表现为 null；这正是我们进行数据质量审计的入口。

6. 第二步代码：数据质量审计，而不是“静默丢数据”

成熟的数据处理程序不能只写一个 `filter()`

然后结束。你应该能回答：原始多少条？每个规则排除了多少条？最终多少条？

```
raw_count = orders.count()

non_paid_count = orders.filter(col("status") != "PAID").count()

paid = orders.filter(col("status") == "PAID")

bad_quantity_count = paid.filter(
    col("quantity").isNull() | (col("quantity") <= 0)
).count()

bad_price_count = paid.filter(
    col("unit_price").isNull() | (col("unit_price") <= 0)
```

```
).count()

base = paid.filter(
    col("quantity").isNotNull() & (col("quantity") > 0) &
    col("unit_price").isNotNull() & (col("unit_price") > 0)
)

unknown_product_count = (base
    .join(products, on="product_id", how="left_anti")
    .count())

print("QUALITY raw_count=", raw_count)
print("QUALITY non_paid_count=", non_paid_count)
print("QUALITY bad_quantity_count=", bad_quantity_count)
print("QUALITY bad_price_count=", bad_price_count)
print("QUALITY unknown_product_count=", unknown_product_count)
```

预期：raw=13, non_paid=2, bad_quantity=1, bad_price=1, unknown_product=1。

这里会触发多个 Action，所以会产生多个

Job。真实大数据项目里可以把多项质量统计合并成一次聚合，减少重复扫描；本讲先用最直观的写法帮助理解。

7. 第三步代码：Broadcast Join、派生 revenue 与复用 cache

products 是很小的维表，因此这里显式 `broadcast(products)`。这表示：把小表复制到 Executor，避免为了 Join 把大订单表按 product_id 进行大规模 Shuffle。

```
clean = (base
    .join(broadcast(products), on="product_id", how="inner")
    .withColumn("revenue", col("quantity") * col("unit_price"))
    .select(
        "order_id", "user_id", "product_id", "product_name", "category",
        "city", "order_date", "quantity", "unit_price", "revenue"
    ))

# 缓存数据
clean.cache()
valid_count = clean.count() # 验证 Action 是否 cache
print("QUALITY final_valid_count=", valid_count)

category_revenue = (clean
    .groupBy("category")
    .agg(
        F.round(F.sum("revenue"), 2).alias("revenue"),
        F.count("*").alias("valid_orders")
    )
    .orderBy(col("revenue").desc()))

city_revenue = (clean
    .groupBy("city")
    .agg(F.round(F.sum("revenue"), 2).alias("revenue"))
    .orderBy(col("revenue").desc()))

category_revenue.show(truncate=False)
city_revenue.show(truncate=False)
```

一次项目运行为什么会产生多个 Job / Stage?



重要：Action 触发 Job；宽依赖/Exchange 往往切 Stage；每个 Stage 内有多个 Task。

cache() 本身不执行。clean.count() 才会真正计算并把分区放入缓存。

后续两个聚合如果复用 clean，可减少“重新读 CSV + 重新清洗 + 重新 Join”的成本。

本讲小数据看不出性能差异，但执行关系与真实 TB 级任务完全相同。

看到 `groupBy()` 时就要想到：相同 category / city 的数据必须重新聚集，通常会出现 Exchange / Shuffle。Broadcast Join 可以减少 Join 本身的 Shuffle，但不能消灭后面的聚合 Shuffle。

8. 第四步代码：同一份 DataFrame 用 SQL 再算一次

这一部分不是为了重复劳动，而是验证第15~16讲：DataFrame API 与 SQL 最终进入同一个 Spark SQL / Catalyst 执行体系。

```
clean.createOrReplaceTempView("sales")
```

```

sql_result = spark.sql("""
SELECT
    category,
    ROUND(SUM(revenue), 2) AS revenue,
    COUNT(*) AS valid_orders
FROM sales
GROUP BY category
ORDER BY revenue DESC
""")

sql_result.show(truncate=False)

print("=== category_revenue physical plan ===")
category_revenue.explain("formatted")

```

阅读 explain 时先找：`Scan` → `BroadcastHashJoin`（若选择广播）→ `HashAggregate` → `Exchange` → `Sort`。其中 `Exchange` 是最值得警惕的分布式边界之一。

9. 第五步代码：写 Parquet，并利用目录分区

CSV 适合交换和查看；Parquet 更适合分析型数据。Spark 官方文档说明 Parquet 是列式格式，Spark SQL 可以读写并保留 Schema。这里把清洗后的明细按 order_date 分区写回 HDFS。

```

clean_out = "hdfs:///user/hadoop/spark19/output/clean_sales"
cat_out = "hdfs:///user/hadoop/spark19/output/category_revenue"
city_out = "hdfs:///user/hadoop/spark19/output/city_revenue"

(clean.write
 .mode("overwrite")
 .partitionBy("order_date")
 .parquet(clean_out))

category_revenue.write.mode("overwrite").parquet(cat_out)
city_revenue.write.mode("overwrite").parquet(city_out)

# 验证数据是否正确
verify = spark.read.parquet(cat_out).orderBy(col("revenue").desc())
verify.show(truncate=False)

clean.unpersist()
spark.stop()

```

不要在真实大数据任务里为了“得到一个文件”而随手 `coalesce(1)`。那会把最终写出压到一个 Task，常常制造新的瓶颈。

10. 完整脚本：lecture19_sales.py

下面是可以直接保存运行的完整版本。前面分块讲过的代码现在合在一起。

```
from pyspark.sql import SparkSession, functions as F
from pyspark.sql.functions import col, broadcast
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType

spark = SparkSession.builder.appName("Lecture19SalesETL").getOrCreate()

orders_schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("user_id", StringType(), True),
    StructField("product_id", StringType(), True),
    StructField("city", StringType(), True),
    StructField("order_date", StringType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("unit_price", DoubleType(), True),
    StructField("status", StringType(), True),
])
products_schema = StructType([
    StructField("product_id", StringType(), True),
    StructField("product_name", StringType(), True),
    StructField("category", StringType(), True),
])

orders = spark.read.option("header", True).schema(orders_schema).csv(
    "hdfs:///user/hadoop/spark19/input/orders.csv")
products = (spark.read.option("header", True).schema(products_schema).csv(
    "hdfs:///user/hadoop/spark19/input/products.csv")
    .dropDuplicates(["product_id"]))

raw_count = orders.count()
non_paid_count = orders.filter(col("status") != "PAID").count()
paid = orders.filter(col("status") == "PAID")
bad_quantity_count = paid.filter(col("quantity").isNull() | (col("quantity") <= 0)).count()
bad_price_count = paid.filter(col("unit_price").isNull() | (col("unit_price") <= 0)).count()
base = paid.filter(
    col("quantity").isNotNull() & (col("quantity") > 0) &
    col("unit_price").isNotNull() & (col("unit_price") > 0))
unknown_product_count = base.join(products, "product_id", "left_anti").count()

print("QUALITY raw_count=", raw_count)
print("QUALITY non_paid_count=", non_paid_count)
print("QUALITY bad_quantity_count=", bad_quantity_count)
print("QUALITY bad_price_count=", bad_price_count)
print("QUALITY unknown_product_count=", unknown_product_count)

clean = (base.join(broadcast(products), "product_id", "inner")
    .withColumn("revenue", col("quantity") * col("unit_price"))
    .select("order_id", "user_id", "product_id", "product_name", "category",
        "city", "order_date", "quantity", "unit_price", "revenue"))
clean.cache()
print("QUALITY final_valid_count=", clean.count())

category_revenue = (clean.groupBy("category")
    .agg(F.round(F.sum("revenue"), 2).alias("revenue"),
        F.count("*").alias("valid_orders"))
    .orderBy(col("revenue").desc()))
city_revenue = (clean.groupBy("city")
    .agg(F.round(F.sum("revenue"), 2).alias("revenue"))
    .orderBy(col("revenue").desc()))

category_revenue.show(truncate=False)
city_revenue.show(truncate=False)
clean.createOrReplaceTempView("sales")
spark.sql("""
SELECT category, ROUND(SUM(revenue), 2) AS revenue, COUNT(*) AS valid_orders
```

```
FROM sales GROUP BY category ORDER BY revenue DESC
""").show(truncate=False)
category_revenue.explain("formatted")

clean.write.mode("overwrite").partitionBy("order_date").parquet(
    "hdfs:///user/hadoop/spark19/output/clean_sales")
category_revenue.write.mode("overwrite").parquet(
    "hdfs:///user/hadoop/spark19/output/category_revenue")
city_revenue.write.mode("overwrite").parquet(
    "hdfs:///user/hadoop/spark19/output/city_revenue")

spark.read.parquet(
    "hdfs:///user/hadoop/spark19/output/category_revenue"
).orderBy(col("revenue").desc()).show(truncate=False)

clean.unpersist()
spark.stop()
```

11. 提交到 YARN：从代码变成真正分布式任务

在单节点教学虚拟机里，资源通常有限，所以先使用保守资源参数。不要照搬生产集群的 Executor 数量。

提交命令：

```
cd ~/spark19
# 在 VS Code 中 lecture19_sales.py 文件所在目录

# 查看 HDFS/YARN 情况
jps
yarn node -list

# 提交到 YARN
spark-submit \
  --master yarn \
  --deploy-mode client \
  --executor-cores 1 \
  --executor-memory 1g \
  --driver-memory 1g \
  lecture19_sales.py
```

client 模式的优点：Driver 日志直接显示在当前终端，初学阶段更容易观察。生产环境是否使用 cluster mode，应根据部署方式和运维要求决定。

11.1 运行时同时观察 YARN

```
# 在 VS Code 中
yarn application -list

# 查看 application_... 的状态
yarn application -status application_xxx

# 查看 application_xxx 的日志
yarn logs -applicationId application_xxx | less
```

12. 验收：程序跑完以后必须检查什么？

第一层是业务结果。终端中的 category_revenue 应与手算完全一致：

category	revenue	有效订单数
Digital	11997	3
Accessory	1290	4
Education	118	1

city	revenue
Guangzhou	6397
Beijing	3316
Shenzhen	2999
Shanghai	693

第二层是数据质量日志：

指标	预期
raw_count	13

指标	预期
non_paid_count	2
bad_quantity_count	1
bad_price_count	1
unknown_product_count	1
final_valid_count	8

第三层是 HDFS 输出结构：

```
hdfs dfs -ls -R /user/hadoop/spark19/output | head -80
```

```
# 检查 clean_sales/order_date=2026-09-01/ 数据
# Parquet 格式输出 hdfs dfs -cat 00"000"0
```

验收结论：结果正确 + 质量计数正确 + HDFS 输出可再次读取，才算这一项目真正闭环。

13. Spark UI：把“程序慢”变成可观察证据

Spark UI：本讲只盯这 6 类证据



本项目数据很小，通常几秒内结束，因此 UI 很难长期观察。可以在调试版本中临时在 `clean.count()` 后加入 `time.sleep(60)`，让应用保持 60 秒；观察完后删除。

```
# 检查 Spark UI 是否出现异常
import time
time.sleep(60)
```

建议观察顺序

位置	你要问的问题
Jobs	有多少个 Action 触发了 Job？
Stages	哪一步出现 Shuffle / Exchange？
Tasks	Task 数是否与 Partition 数对应？
SQL	BroadcastHashJoin / HashAggregate / Exchange 是否出现？
Storage	clean.cache() 后是否出现缓存占用？

位置	你要问的问题
Executors	是否存在 Spill、GC 异常或 Lost Executor？

一次项目运行为什么会产生多个 Job / Stage？



重要：Action 触发 Job；宽依赖/Exchange 往往切 Stage；每个 Stage 内有多个 Task。

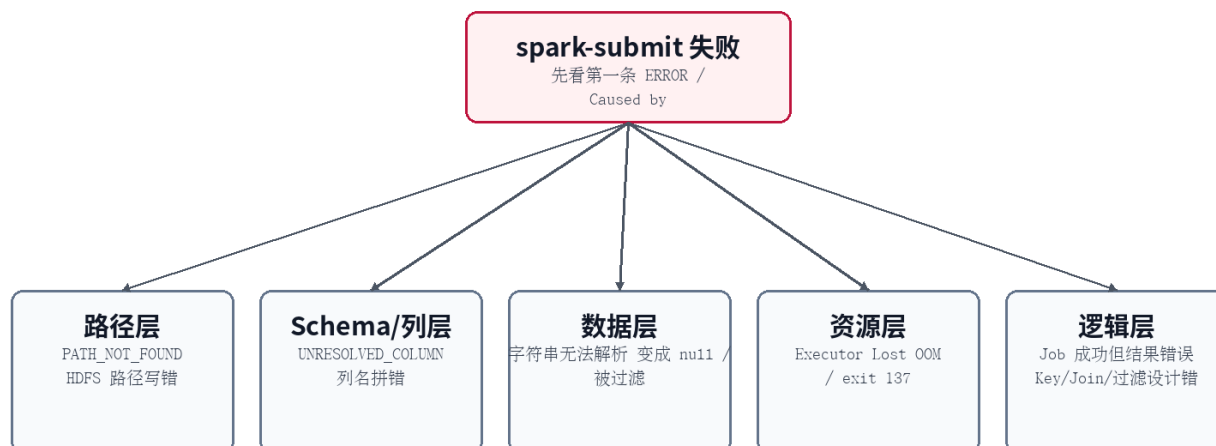
cache() 本身不执行。clean.count() 才会真正计算并把分区放入缓存。

后续两个聚合如果复用 clean，可减少“重新读 CSV + 重新清洗 + 重新 Join”的成本。

本讲小数据看不出性能差异，但执行关系与真实 TB 级任务完全相同。

14. 故意制造 4 个错误：学会从错误反推层级

故障排查树：先判断失败发生在哪一层



原则：不要从“改内存参数”开始。先定位层级，再做最小修改。

错误 1：HDFS 路径写错

```
# □ input/orders.csv □□□□ input/orderz.csv
# □□□□PATH_NOT_FOUND / Path does not exist
# □□□□□□ hdfs dfs -ls□□□□□□
```

错误 2：列名写错

```
# □ col("quantity") □□ col("quantity")
# □□□□UNRESOLVED_COLUMN / cannot resolve ...
# □ printSchema()□□□□□ executor-memory□
```

错误 3：脏数据没有显式处理

如果你直接计算 revenue 而不检查 unit_price 是否为 null，O013 会把错误静默带入后续。Schema 只帮你“识别类型问题”，不会替你定义业务规则。

错误 4：资源问题

真实数据扩大后，如果出现 Executor Lost、Container killed、exit code 137，再去检查 Executor 内存、memory overhead、Shuffle/Spill、数据倾斜等。资源问题必须有日志和 UI 证据，不能靠猜。

15. 为什么本项目这样设计？把前18讲串回来

设计选择	背后的原理
显式 Schema	Spark 能理解列类型；非法数值变成可审计问题
先过滤再 Join	减少进入后续计算的数据量
Broadcast products	小维表广播，避免大表 Join Shuffle
clean.cache()	后续多个 Action / 聚合复用同一份计算结果
groupBy + agg	按 Key 重分布，会引入 Shuffle / Exchange
Parquet	列式、保留 Schema，适合分析型读取
partitionBy(order_date)	把日期编码到目录层级，便于分区裁剪思维
不 coalesce(1)	避免人为制造单 Task 写出瓶颈
手算 + 质量计数 + UI	正确性、数据质量、执行行为三类证据闭环

你现在已经能看到：HDFS 负责存；YARN 负责资源；Spark SQL/Catalyst 负责计划；Executor/Task 负责执行；Shuffle 负责跨分区重组；Parquet 负责高效分析存储。它们不再是零散名词，而是一条工作链。

16. 练习：不要看答案，先自己改

- 练习1：增加“每天各类别销售额”统计。你准备 groupBy 哪两个字段？
- 练习2：求销售额最高的 2 个城市。你会用 orderBy + limit，还是 groupByKey？为什么？
- 练习3：假设 products 从 5 行变成 5 TB，还应该 broadcast 吗？为什么？
- 练习4：把 clean.cache() 删除，重新推理后续两个聚合会发生什么。
- 练习5：为什么输出 Parquet 后，不应该用 hdfs dfs -cat part-xxxx 直接验证内容？
- 练习6：如果 category=Digital 占全部数据 95%，你会从 Spark UI 的哪些现象判断数据倾斜？

参考答案

- 1：groupBy("order_date", "category")；因为同一天、同类别的数据必须进入同一聚合组。
- 2：先按 city 聚合，再 orderBy(revenue desc).limit(2)。groupByKey 不是“取前2”的必要工具，而且通常带来更高 Shuffle/内存成本。
- 3：不应该。广播适用于足够小、能够被 Executor 复制和保存的关系；5 TB 维表会让广播完全不可行。
- 4：每个 Action 需要根据 Lineage 重新获取或计算 clean 上游；是否完全重算取决于执行计划与可复用结果，但不能再依赖显式缓存复用。
- 5：Parquet 是二进制列式格式，不是文本。应使用 Spark/Parquet Reader 重新读取并 show/count。
- 6：少数 Task Duration 明显更长、Shuffle Read/Records 远高于其他 Task、Stage 长尾明显。

17. 本讲最小闭环能力

完成本讲后，你至少应该能够独立完成下面这条链：

```
□□ HDFS/YARN
→ □□ CSV □□□ HDFS
→ □□ Schema
→ □□□□□□
→ DataFrame □□
→ Broadcast Join
→ groupBy/SQL □□
→ explain() □□□□□
→ cache □□□ Action/Job
→ Parquet □□ HDFS
→ spark-submit □□ YARN
→ □ Spark UI / YARN □□
→ □□□□□□
→ □□□□□□
```

如果这条链能在没有 AI 帮你敲每一条命令的情况下独立跑通，你就已经具备 Spark 的“最低闭环能力”。后续遇到更复杂项目，本质是在这条链上扩展。

官方资料核对

本讲涉及的 Parquet 读写、spark-submit、YARN 运行方式与 Spark Web UI 观察逻辑，按 Apache Spark 3.5.7 官方文档核对。课程保持 Spark 3.5.x / JDK 8 兼容教学思路，不在此处切换到 Spark 4.x。

主题	官方文档
Parquet	Apache Spark 3.5.7 - Parquet Files
提交应用	Apache Spark 3.5.7 - Submitting Applications
Spark on YARN	Apache Spark 3.5.7 - Running Spark on YARN
监控/UI	Apache Spark 3.5.7 - Monitoring and Instrumentation

下一讲（第20讲）将做整个阶段的最终综合收束：HDFS + YARN + MapReduce + Spark + RDD + DataFrame + Shuffle + DAG + Stage + Task 全部放进一张统一知识地图，然后自然过渡到 Flink。